AI-Assisted Coding
Lab Assignment - 21.1

Name: Zobiya Fatima
Roll No: 2303A51879
Batch: 14

Lab 21 – Collaborative AI Coding: Pair Programming and Version Control Integration
Task 1 – AI-Assisted Pair Programming
• Use AI to generate an initial program (e.g., a simple To - Do List Manager).
• Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).
• Compare the original and improved versions to understand the role of AI in collaboration.
Initial Code:

task1.py > ...

```python
tasks = []
def show_menu():
    print("\n1. Add Task")
    print("2. View Tasks")
    print("3. Delete Task")
    print("4. Exit")
while True:
    show_menu()
    choice = input("Enter choice: ")
    if choice == '1':
        task = input("Enter task: ")
        tasks.append(task)
    elif choice == '2':
        for i, task in enumerate(tasks):
            print(i + 1, task)
    elif choice == '3':
        index = int(input("Enter task number to delete: ")) - 1
        if 0 <= index < len(tasks):
            tasks.pop(index)
    elif choice == '4':
        break
```

```
PS C:\AI Assisted Coding\LAB\LAB-21> python task1.py
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter choice: 2
```

Prompt:
Improve the To-Do List program by:
- Adding error handling
- Validating user input
- Improving user interface messages
- Using functions for better structure

Improved Code:

```
task1.py > ...
tasks = []
def show_menu():
    print("\n===== TO-DO LIST =====")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Delete Task")
    print("4. Exit")
def add_task():
    task = input("Enter task: ").strip()
    if task:
        tasks.append(task)
        print("✅ Task added successfully!")
    else:
        print("❌ Task cannot be empty!")
def view_tasks():
    if not tasks:
        print("📭 No tasks available.")
    else:
        print("\n📋 Your Tasks:")
        for i, task in enumerate(tasks, 1):
            print(f"{i}. {task}")
def delete_task():
    try:
        view_tasks()
        index = int(input("Enter task number to delete: ")) - 1
        if 0 <= index < len(tasks):
            removed = tasks.pop(index)
            print(f"🗑️ Removed: {removed}")
        else:
            print("❌ Invalid task number!")
    except ValueError:
        print("❌ Please enter a valid number!")
while True:
    show_menu()
    choice = input("Enter choice: ")
    if choice == '1':
        add_task()
    elif choice == '2':
        view_tasks()
    elif choice == '3':
        delete_task()
    elif choice == '4':
        print("👋 Exiting program...")
        break
    else:
        print("❌ Invalid choice! Try again.")
```

Output:

```
===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter choice: 1
Enter task: reading
✅ Task added successfully!

===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter choice: 1
Enter task: meditation
✅ Task added successfully!

===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter choice: 2

📋 Your Tasks:
1. reading
2. meditation

===== TO-DO LIST =====
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter choice:
```

Comparison:

Feature	Initial Code	Improved Code
Error Handling	No	Yes
Structure	Basic	Modular
User Experience	Poor	Good
Validation	No	Yes

Observation:

- The initial AI-generated program was simple and worked but lacked error handling and validation.
- After asking AI for improvements, the code became more structured using functions.
- Error handling (try-except) and input validation made the program more reliable.
- The user interface improved with clear messages, making it easier to use.
- Overall, AI helped in both writing and enhancing the code like a pair programmer.

Task 2 – Version Control Integration

• Initialize a Git repository for your project.

• Perform the following steps:

1. Create and switch to a new branch.
2. Add AI-assisted modifications.
3. Commit with a meaningful message.
4. Merge changes back into the main branch.

• Reflect on how version control supports collaborative development with AI assistance.

Code:

```
task1.py > ...
You, now | 1 author (You)
tasks = []
def show_menu():
    print("\n===== TO-DO LIST =====")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Delete Task")
    print("4. Exit")
def add_task():
    task = input("Enter task: ").strip()
    if task:
        tasks.append(task)
        print("✅ Task added successfully!")
    else:
        print("❌ Task cannot be empty!")
def view_tasks():
    if not tasks:
        print("📭 No tasks available.")
    else:
        print("\n📋 Your Tasks:")
        for i, task in enumerate(tasks, 1):
            print(f"{i}. {task}")
def delete_task():
    try:
        view_tasks()
        index = int(input("Enter task number to delete: ")) - 1
        if 0 <= index < len(tasks):
            removed = tasks.pop(index)
            print(f"🗑️ Removed: {removed}")
        else:
            print("❌ Invalid task number!")
    except ValueError:
        print("❌ Please enter a valid number!")
while True:
    show_menu()
    choice = input("Enter choice: ")
    if choice == '1':
        add_task()
    elif choice == '2':
        view_tasks()
    elif choice == '3':
        delete_task()
    elif choice == '4':
        print("👋 Exiting program...")
        break
    else:
        print("❌ Invalid choice! Try again.")
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-21> git add task1.py
PS C:\AI Assisted Coding\LAB\LAB-21> git commit -m "Added error handling, validation, and modular functions"
[improved-version 89d9466] Added error handling, validation, and modular functions
 1 file changed, 37 insertions(+), 12 deletions(-)
PS C:\AI Assisted Coding\LAB\LAB-21> git checkout main
Switched to branch 'main'
PS C:\AI Assisted Coding\LAB\LAB-21> git merge improved-version
Updating eb3e2af..89d9466
 1 file changed, 37 insertions(+), 12 deletions(-)
PS C:\AI Assisted Coding\LAB\LAB-21> git checkout main
Switched to branch 'main'
PS C:\AI Assisted Coding\LAB\LAB-21> git merge improved-version
Updating eb3e2af..89d9466
Switched to branch 'main'
PS C:\AI Assisted Coding\LAB\LAB-21> git merge improved-version
Updating eb3e2af..89d9466
PS C:\AI Assisted Coding\LAB\LAB-21> git merge improved-version
Updating eb3e2af..89d9466
Fast-forward
 task1.py | 49 ++++++
```

Observation:

- Git successfully tracked all changes made in different versions of the program.
- Branching helped to safely add AI-generated improvements without affecting the main code.
- The merge operation combined improved code into the main branch efficiently.
- The fast-forward merge showed there were no conflicts between versions.
- Version control makes development organized and supports collaboration with AI tools.

Task 3 – AI for Commit Message Generation

• Use AI to generate concise, meaningful commit messages based on the changes made.

• Compare AI-generated messages with human-written ones.

• Discuss how AI can improve documentation and traceability in Git history.

Code:

```python
import re

username = input("Enter username: ")
password = input("Enter password: ")

if not re.match("^[a-zA-Z0-9]+$", username):
    print("Invalid username format")
elif username == "admin" and password == "1234":
    print("Login successful")
else:
    print("Invalid credentials")

#Generate a concise Git commit message for the following changes:
#- Added input validation using regex
#- Improved login security
#- Prevented invalid username formats

"Add regex validation for username and enhance login security"

"Updated login.py with validation"      You, now • Uncommitted changes

```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-21> git add login.py
PS C:\AI Assisted Coding\LAB\LAB-21> git commit -m "Add regex-based input validation for username to improve login security"B\LAB-21>
[main c5dc801] Add regex-based input validation for username to improve login security
 1 file changed, 11 insertions(+)
 create mode 100644 login.py
PS C:\AI Assisted Coding\LAB\LAB-21>
```

Observation:

- The AI-generated commit message clearly describes both the change and its purpose.
- It is more detailed compared to the human-written message, which is vague.
- AI improves consistency and readability of commit history.
- It helps developers understand changes quickly without reading the full code.
- Therefore, AI enhances documentation and traceability in Git.

Task 4 – Pair Programming Simulation

• Student A writes the initial version of a program.
• Student B, with AI assistance, suggests improvements or adds features.
• Both students collaborate using Git branching and merging to integrate their work.

Student A creates initial program

```python
# calculator.py (Initial Version by Student A)

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

print("Addition:", add(5, 3))
print("Subtraction:", subtract(5, 3))
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-21> git init
Reinitialized existing Git repository in C:/AI Assisted Coding/LAB/LAB-21/.git/
PS C:\AI Assisted Coding\LAB\LAB-21> git add calculator.py
PS C:\AI Assisted Coding\LAB\LAB-21> git commit -m "Initial calculator with add and subtract functions"
[main 910163b] Initial calculator with add and subtract functions
 1 file changed, 10 insertions(+)
 create mode 100644 calculator.py
PS C:\AI Assisted Coding\LAB\LAB-21> git branch feature-improvements
PS C:\AI Assisted Coding\LAB\LAB-21> git checkout feature-improvements
M       login.py
M       task1.py
Switched to branch 'feature-improvements'
```

Prompt:

Improve this Python calculator program by:

- Adding multiplication and division
- Adding input from user
- Handling division by zero
- Making it menu-driven

Student B uses AI for improvements:

```python
# Improved Version by Student B
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    return a / b
while True:
    print("\n1.Add 2.Subtract 3.Multiply 4.Divide 5.Exit")
    choice = input("Enter choice: ")
    if choice == '5':
        break
    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))
    if choice == '1':
        print("Result:", add(a, b))
    elif choice == '2':
        print("Result:", subtract(a, b))
    elif choice == '3':
        print("Result:", multiply(a, b))
    elif choice == '4':
        print("Result:", divide(a, b))
    else:
        print("Invalid choice")
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-21> git add calculator.py
>>
PS C:\AI Assisted Coding\LAB\LAB-21> git commit -m "Added multiply, divide, user input and error handling"
PS C:\AI Assisted Coding\LAB\LAB-21> git add calculator.py
>>
PS C:\AI Assisted Coding\LAB\LAB-21> git commit -m "Added multiply, divide, user input and error handling"
[feature-improvements d17c6a1] Added multiply, divide, user input and error handling
 1 file changed, 29 insertions(+), 3 deletions(-)
PS C:\AI Assisted Coding\LAB\LAB-21> git checkout main
M       login.py
M       task1.py
Switched to branch 'main'
PS C:\AI Assisted Coding\LAB\LAB-21> git merge feature-improvements
Updating 910163b..d17c6a1
Fast-forward
 calculator.py | 32 ++++++++++++++++++++++++++++---
 1 file changed, 29 insertions(+), 3 deletions(-)
PS C:\AI Assisted Coding\LAB\LAB-21>
```

Observation:
In this task, we simulated pair programming where one student developed the initial code and another improved it using AI assistance. Git branching and merging helped integrate both contributions effectively. AI significantly enhanced productivity by suggesting features and improving code quality, making collaboration faster and more efficient.

Task 5 – AI-Assisted Debugging
• Create a program with logical or syntax errors.
• Use AI to identify and fix the errors.
• Compare the buggy and corrected versions.

Code with error:

```
task5.py > ...
def calculate_average(numbers):
    total = 0
    for i in range(len(numbers)):
        total = total + numbers[i]

    avg = total / len(numbers)
    return avg

nums = [10, 20, 30, 40, 50]

print("Average is: " + calculate_average(nums))
```

```
PS C:\AI Assisted Coding\LAB\LAB-21> python task5.py
Traceback (most recent call last):
  File "C:\AI Assisted Coding\LAB\LAB-21\task5.py", line 11, in <module>
    print("Average is: " + calculate_average(nums))
          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
TypeError: can only concatenate str (not "float") to str
PS C:\AI Assisted Coding\LAB\LAB-21>
```

Prompt:
I have written a Python program to calculate the average of numbers, but it has errors. Please identify both syntax and logical errors, explain them clearly, and provide a corrected version of the code.

Corrected Code:

```
task5.py > ...
def calculate_average(numbers):
    if len(numbers) == 0:
        return 0

    total = sum(numbers)
    avg = total / len(numbers)
    return avg

nums = [10, 20, 30, 40, 50]

print("Average is:", calculate_average(nums))
```

```
PS C:\AI Assisted Coding\LAB\LAB-21> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-21/task5.py"
Average is: 30.0
PS C:\AI Assisted Coding\LAB\LAB-21>
```

Observation: AI helped to quickly identify and fix errors in the program. It improved code quality and reduced debugging time. It also helped in understanding mistakes easily.

Task 6 – Feature Expansion using AI

- Develop a basic application (e.g., Calculator).
- Use AI to add advanced features (history, validation).
- Analyze improvements.

Basic Code:

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
op = input("Enter operator (+, -, *, /): ")

if op == "+":
    print("Result:", a + b)
elif op == "-":
    print("Result:", a - b)
elif op == "*":
    print("Result:", a * b)
elif op == "/":
    print("Result:", a / b)
else:
    print("Invalid operator")
```

```
PS C:\AI Assisted Coding\LAB\LAB-21> python calculator_basic.py
Enter first number: 77
Enter second number: 55
Enter operator (+, -, *, /): *
Result: 4235.0
PS C:\AI Assisted Coding\LAB\LAB-21>
```

Prompt:

I have a basic Python calculator program.

Please enhance it by:

1. Adding input validation (handle invalid numbers and division by zero)
2. Adding a feature to store and display calculation history
3. Making the program user-friendly with clear messages

Also provide the improved version of the code.

Improved Code:

```python
history = []
def get_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Invalid input! Please enter a number.")
def calculator():
    while True:
        print("\n--- Calculator ---")
        print("1. Perform Calculation")
        print("2. View History")
        print("3. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            a = get_number("Enter first number: ")
            b = get_number("Enter second number: ")
            op = input("Enter operator (+, -, *, /): ")
            if op == "+":
                result = a + b
            elif op == "-":
                result = a - b
            elif op == "*":
                result = a * b
            elif op == "/":
                if b == 0:
                    print("Error! Division by zero.")
                    continue
                result = a / b
            else:
                print("Invalid operator!")
                continue
            output = f"{a} {op} {b} = {result}"
            print("Result:", result)
            history.append(output)
        elif choice == "2":
            print("\n--- History ---")
            if not history:
                print("No calculations yet.")
            else:
                for item in history:
                    print(item)
        elif choice == "3":
            print("Exiting...")
            break
        else:
            print("Invalid choice!")
calculator()
```

Output:

```
--- Calculator ---
1. Perform Calculation
2. View History
3. Exit
Enter choice: 1
Enter first number: 88
Enter second number: 56
Enter operator (+, -, *, /): *
Result: 4928.0

--- Calculator ---
1. Perform Calculation
2. View History
3. Exit
Enter choice: 2

--- History ---
88.0 * 56.0 = 4928.0

--- Calculator ---
1. Perform Calculation
2. View History
3. Exit
Enter choice: 3
Exiting...
PS C:\AI Assisted Coding\LAB\LAB-21>
```

Observation:

The basic calculator performs simple operations but lacks error handling and additional features. Using AI, we enhanced it by adding input validation and a history feature. The improved version is more user-friendly and reliable. This shows how AI helps in quickly upgrading applications with better functionality.